Trabajo Práctico Integrador

Calidad de Software aplicada al Trabajo Final de Grado: **VoluntariAR**

Integrantes	Santiago Planas Montilla — legajo 2111770
	Santiago Francisco Antonio Ruibal — legajo 2009644
Docente	Pablo Alejandro Virgolini
Nivel declarado	**Nivel 3 — prototipo avanzado**
Fecha de presentación	26 de octubre de 2026
Repositorio	github.com/SantiagoPlanasM/voluntariar-PF

Todas las cifras de este informe provienen de ejecuciones del propio proyecto (GitHub Actions, SonarCloud, k6 y el registro de defectos en GitHub Issues), con la fuente indicada en cada caso. Corte de datos: 17/10/2026. Último cambio de código: `df529c3` (CI #19: tests y umbrales de cobertura del frontend); los commits posteriores son solo de documentación.

Índice

1. Resumen y nivel declarado
2. Metas de calidad: cumplidas y no cumplidas
3. Los ejes del trabajo (8 temas + documentación)
4. Evolución de las métricas
5. Análisis crítico
6. Conclusiones
7. Anexo: entregables, cómo verificar cada número y uso de IA

1. Resumen y nivel declarado

VoluntariAR es una red social de voluntariado que conecta voluntarios, ONG y empresas. Backend en Node 22 + Express con SQL a mano (SQLite en desarrollo y pruebas, Postgres en producción); frontend en React 18 + Vite; desplegado en Render (plan gratuito). La **integración externa** evaluada es **Mercado Pago** (donaciones con Checkout Pro, en modo prueba); también usa Cloudinary (archivos) y Resend (emails).

Se declaró el **nivel 3 (prototipo avanzado)**, porque el producto estaba funcionalmente terminado y desplegado. El trabajo agregó lo que faltaba para que la calidad se apoye en datos: **pipeline de CI/CD** con quality gate, **SonarCloud** como herramienta nueva, **deploy automático con smoke y E2E contra Mercado Pago**, **pruebas de stress y esfuerzo sostenido**, plan de pruebas versionado y **registro de defectos en GitHub Issues** con trazabilidad.

> **En números:** 808 tests de backend en SQLite y 810 en Postgres (UTC y zona horaria de Córdoba), 864 de frontend, todos en verde (CI #19). Cobertura del backend 92,95 % de líneas; del frontend, 90,11 % de líneas y más del 80 % en sentencias, ramas y funciones. SonarCloud: issues abiertas **157** → **63**, bugs **39** → **0**, vulnerabilidades **18** → **0** abiertas. 2 ejecuciones fallidas reales del CI, ambas corregidas en commits posteriores. 18 defectos registrados con pasos, esperado, obtenido, commit y prueba de regresión. 32 commits durante el TP.

2. Metas de calidad: cumplidas y no cumplidas

Cada aspecto de la tabla del enunciado, contra lo exigido para el nivel 3.

Aspecto	Exigido	Logrado (evidencia)	Estado
Pipeline	Compilación, pruebas, análisis estático, cobertura, quality gate y reporte automático	`ci.yml`: build y tipos del frontend, tests (SQLite y Postgres × UTC y Córdoba), cobertura con umbral, SonarCloud + gate, `npm audit`; resumen y artefactos por run. CI #15 en verde	**Cumplida**
Quality gate	Bloquea ante bugs críticos, duplicación > 5 % y cobertura del código nuevo < 80 %	`sonar-gate.js` (también bloquea vulnerabilidades críticas). Bloqueó de verdad en el **CI #7** (2 bugs críticos). Hoy: 0 / 0 / 2,2 % / 89,9 %	**Cumplida**
Cobertura del código nuevo	80 %	**89,9 %** (SonarCloud)	**Cumplida**
Plan de pruebas y riesgos	Versionado, criterios de entrada y salida, riesgos P × I, trazabilidad requisito → caso → defecto	`PLAN_PRUEBAS.md` v0.9 (9 versiones en git), 10 riesgos priorizados, criterios, matriz ISO 25010; `REGISTRO_DEFECTOS.md`	**Cumplida**

Aspecto	Exigido	Logrado (evidencia)	Estado
Defectos	Con pasos, esperado y obtenido, vinculados a su caso, en Jira u otra	18 Issues en GitHub (#2–#19) con plantilla, labels, caso, commit del arreglo y prueba de regresión	**Cumplida**
Herramienta nueva	Proceso de 5 pasos	SonarCloud: requisitos, mercado (4 alternativas), prueba de concepto (PR #1), matriz (92 contra 84 de Codacy), despliegue incremental	**Cumplida**
Cobertura del código existente	80 % (nivel 3)	Backend: líneas 92,95 %, ramas 84,57 %, funciones 90,59 %. Frontend (CI #19): líneas **90,11 %**, sentencias **86,36 %**, ramas **81,18 %** y funciones **82,11 %**, con umbral bloqueante en las cuatro. En el CI #15 ramas y funciones no llegaban (74,08 % y 67,99 %)	**Cumplida**
Smoke test	Ejecutado por el pipeline tras el despliegue	`deploy.yml` → `post-deploy.js smoke` después de cada deploy (Deploys #1 a #8: smoke en verde en todos)	**Cumplida**
Integración externa	Flujo E2E automatizado en cada despliegue	En cada deploy: login → donación → **preferencia creada en Mercado Pago real** → estado pendiente. **Sin el pago**: no es automatizable en modo prueba (403 de CloudFront fuera de un navegador). El pago completo, verificado a mano 3 veces	**Cumplida, con un límite**
Stress y esfuerzo sostenido	Stress ejecutado, con resultados y umbrales	Workflow "Carga" (k6): stress p95 3,06 ms (umbral 500), sostenido p95 3,17 ms (umbral 300), producción p95 418,90 ms (umbral 2000); 0 % de errores	**Cumplida**

3. Los ejes del trabajo

3.1 Introducción a la calidad: objetivos y contingencia

13 objetivos medibles agrupados por **usuario** (rendimiento, deploys verificados, seguridad, accesibilidad), **equipo** (cero regresiones, cobertura, regresión por defecto, tiempo de recuperación) y **organización** (costo $0, dependencias sin vulnerabilidades, defectos escapados, continuidad de la demo). Cada uno lleva la práctica que lo sostiene y su valor medido (`PLAN_PRUEBAS.md §2.1`). El **plan de contingencia** por dependencia (`§3`) documenta el comportamiento real del código: Mercado Pago y Cloudinary responden 503 sin configuración y cortan a los 20 s con 502; Resend se omite sin romper; ante Render dormido, se despierta con `/health` o se usa el plan B local.

3.2 Fundamentos del testing

- **Arrange–Act–Assert y criterio previo:** los tests nuevos marcan los tres bloques y llevan el criterio de aceptación en el encabezado, escrito antes del código (por ejemplo, `sonar-gate.test.js`, `donation-amount.test.js`).
- **Defecto real con contraste rojo/verde:** B7, el filtro de palabras que buscaba subcadenas. El mismo test de regresión corrido en worktrees aparte: sobre el commit **anterior al arreglo**, 2/2 en rojo ("computadora" → 400, "imbécil" → 201); **sobre el arreglo**, 2/2 en verde. Evidencia: `docs/calidad/evidencia/b7-*.txt`.
- **Los siete principios**, con un ejemplo del proyecto cada uno (`PLAN_PRUEBAS.md §9`). El más visible: "*las pruebas muestran la presencia de defectos, no su ausencia*". Con la suite en verde, 4 defectos aparecieron recién en producción.
- **Mutation testing** sobre todo el código nuevo, con corrida de control. Encontró huecos reales en los tests: por ejemplo, el mínimo de 8 caracteres de la contraseña no tenía prueba de borde (D14).

3.3 Modelos de calidad y niveles de prueba

Matriz ISO/IEC 25010 (8 características) con evidencia y brecha de cada una (`PLAN_PRUEBAS.md §5`). **Top 3: seguridad** (CV privado, borrado de cuenta, Ley 25.326), **fiabilidad** (donaciones y cupos sin doble acreditación) y **compatibilidad** (la diferencia SQLite/Postgres fue la fuente de defectos más frecuente: B1, B2, B8, A6). **Mapa de niveles:** unitario, integración de API contra base real, componentes, sistema (smoke), integración externa (E2E), no funcional (carga) y aceptación (guion de la demo). **Despliegue verificado en cada instancia:** local y CI (suites en cada push) y producción (smoke + E2E en cada deploy; `/health` informa el commit desplegado).

3.4 Métricas y análisis estático

SonarCloud en el pipeline (`METRICAS.md`): 17.083 líneas de código; duplicación 2,2 %; complejidad ciclomática 4.045 y cognitiva 2.720 (la más alta: `routes/projects.js`, 333); deuda 126 min; calificaciones A/A/A. **Métrica de proceso:** tasa de éxito del pipeline (CI 86,7 %, Deploy 62,5 %) y tiempo hasta volver a verde (5,1 y 30 min). **Métrica de proyecto:** densidad de pruebas, con 145,5 tests/KLOC en el backend y 60,6 en el frontend. **Mejora medida:** la evolución de SonarCloud de la sección 4.

3.5 Pruebas dinámicas

`DISENO_CASOS.md`:

- **Monto de la donación:** 5 clases de equivalencia y valores límite 99/100/101 y 499.999/500.000/500.001.
- **Contraseña:** bordes de 7/8 caracteres y de 72/73 bytes, también con ñ.
- **Acceso al CV:** tabla de decisión con 10 reglas, cada una con su test. Se agregó la que faltaba.
- **Cobertura de sentencia y de decisión** de `amountError`: 100 % de las dos con todas las clases. Con un solo caso válido se ejecutan 6/6 líneas pero solo 1 de 4 ramas: es el ejemplo de por qué una no implica la otra.

3.6 Aseguramiento de la calidad

Plan de pruebas versionado (v0.1 → v0.9) con criterios de entrada, salida y suspensión; **10 riesgos** priorizados por probabilidad × impacto (los más altos: Render dormido en la demo = 9; consultas que fallan solo en Postgres y la carga que tira el servicio gratuito = 6). **QA** (las 24 reglas de `AI_RULES.md`, tests primero, revisiones de código, gate) distinguido de **QC** (las suites, el smoke, la carga, la verificación en producción). **Plantillas de caso y de defecto** (`PLAN_PRUEBAS.md §10` y `.github/ISSUE_TEMPLATE/defecto.md`) y **trazabilidad** requisito → caso → defecto → Issue (`REGISTRO_DEFECTOS.md`).

3.7 Automatización del testing

- **Pirámide:** la base es ancha en integración de API contra base real (donde aparecieron casi todos los defectos), con unitarias, componentes, smoke, E2E y carga encima.
- **Pruebas de API**: 60 archivos de integración. **Integración con la plataforma externa**: contrato y dobles, prueba real a mano y E2E en cada deploy (`INTEGRACION_MERCADOPAGO.md`).
- **CI/CD:** `ci.yml` → `deploy.yml` (deploy hooks de Render, espera del commit en `/health`, smoke y E2E), con el auto-deploy de Render apagado.
- **Una prueba de regresión por cada defecto corregido:** las 18 de `REGISTRO_DEFECTOS.md`, todas en el CI.

3.8 Herramientas del ecosistema

Mapa de herramientas por propósito (`HERRAMIENTA_NUEVA.md`): node:test, Vitest + Testing Library, TypeScript 7, Postgres en Docker o como servicio, GitHub Actions, SonarCloud, k6, `npm audit`, GitHub Issues + `gh`, Chrome. **Herramienta nueva:** SonarCloud, con el proceso de 5 pasos y la matriz con pesos (SonarCloud 92, Codacy 84, Qlty 72, ESLint + jscpd 65). **Registro de defectos:** GitHub Issues. **Stress y esfuerzo sostenido:** k6 2.3.0 en el workflow "Carga".

3.9 Documentación (transversal)

Cada cambio dejó su sección en `PROJECT_ANALYSIS.md` (§183–§207), con qué se hizo, verificación, **errores propios** y **no verificado**. Los documentos del TP están en `docs/calidad/` (anexo).

4. Evolución de las métricas

4.1 SonarCloud (historial de análisis del pipeline)

Análisis	Bugs	Vulnerab.	Code smells	Deuda	Fiabilidad / seguridad	Qué pasó
`77f4fdf` (CI #7)	39	18	103	416 min	E / D	Primer análisis de todo el proyecto: el gate bloqueó (2 bugs críticos)
`959f74b` (CI #8)	37	17	103	416 min	C / C	Corregidos los críticos y la firma con SHA-1
`2a53138` (CI #14)	1	5	103	416 min	C / C	36 promesas revisadas; ReDoS e inyección en logs corregidos
`80042fb` (CI #15)	1	5	71	256 min	C / C	Accesibilidad: 32 issues
Después del triage	**0**	**0**	**63**	**126 min**	**A / A**	14 aceptadas con justificación; quedan solo cosméticas

4.2 Pruebas y cobertura

Momento	Backend SQLite / Postgres	Frontend	Cobertura backend (líneas)	Cobertura frontend (líneas)
Inicio del TP (17/10, local)	782 / 784	694	92,94 % (sin medir en CI)	sin medir
CI #3 (primera medición correcta)	782 / 784	694	92,94 %	83,14 % (CI #2)
CI #15	808 / 810	699	92,95 %	83,26 %
CI #19 (cobertura del frontend, §209–§212)	**808 / 810**	**864**	**92,95 %**	**90,11 %**

El CI #2 informó 96,51 % para el backend porque Node 22 medía otros archivos. Se detectó al comparar el pipeline con la corrida local y se corrigió en el CI #3 (defecto D09). Los 808 incluyen un archivo auxiliar que el runner cuenta como test: los tests reales son 807.

4.3 Pipeline

Workflow	Runs	En verde	Fallidos y causa
CI	15	13 (86,7 %)	#4: un comando que no corría los tests del gate. #7: el gate encontró 2 bugs críticos. Ambos **defectos reales**, corregidos en commits posteriores en 5 y 30 minutos
Deploy	8	5 (62,5 %)	#1–#3: configuración de la cuenta de E2E (401, rol de ONG, email). Los detectó el E2E
Carga	1	1	—

4.4 Rendimiento (Carga #1, k6)

Prueba	p95	Errores	Umbral
Stress hasta 50 usuarios (runner)	3,06 ms	0 %	< 500 ms ✓
Stress de 50 a 100 usuarios (sin umbral)	4,56 ms	0 %	—
Esfuerzo sostenido, 20 usuarios × 10 min	3,17 ms	0 %	< 300 ms ✓

Prueba	p95	Errores	Umbral
Producción (Render gratis), 10 usuarios × 2 min	418,90 ms	0 %	< 2 s ✓

5. Análisis crítico

- **Las herramientas encontraron lo que las pruebas no.** Con 782 tests en verde, SonarCloud encontró una **denegación de servicio real**: un email de 80.000 caracteres bloqueaba toda la API 3,4 s por pedido (reproducido y corregido con un tope antes del regex). También encontró una **inyección en los logs** desde el webhook de Mercado Pago. Y una de sus sugerencias (`localeCompare` para ordenar) habría roto la firma de Cloudinary: el análisis estático hay que leerlo con criterio, no obedecerlo.
- **Los defectos escapados vienen de lo que los dobles no ven.** Los 4 defectos encontrados en producción salieron de flujos con un servicio real o de lo que ve otro rol (el panel de la ONG después de una donación real). De ahí el E2E contra Mercado Pago en cada deploy. La tasa (22,2 %) es sobre una **selección** de defectos: con todos los hallazgos de las revisiones sería menor.
- **El frontend era la deuda.** En el CI #15 la cobertura de ramas (74 %) y de funciones (68 %) del frontend no llegaba al 80 %. Con un test de contrato del cliente HTTP, tests de componentes y el borrado de código muerto pasó a 81,18 % y 82,11 % (CI #19). Su densidad de pruebas subió de 60,6 a 75,9 tests/KLOC, que sigue siendo cerca de la mitad de la del backend. Los componentes más complejos (Explorar, el detalle del proyecto, el mapa) son los menos probados en sus caminos de error.
- **No se encontró el punto de quiebre.** Con lecturas y los datos de demo, la API no se degradó hasta 100 usuarios. Encontrarlo exige más usuarios, más datos o escrituras, que se dejaron afuera a propósito para no crear datos ni gatillar los limitadores.
- **Producción es unas 100 veces más lenta que el runner** (mediana 103 ms contra 1,4 ms). Es esperable en el plan gratuito, pero no se midió qué parte es CPU, base de datos o red.
- **El proceso tuvo errores propios, y quedaron documentados:** un comando del CI que no corría tests (el primer run fallido), una cobertura inflada en el pipeline, un test de tiempo inestable, un registro de defectos que al principio dejaba afuera 3 escapados, y el orden del proceso de la herramienta nueva (se eligió con el borrador de la matriz). Cada uno está en `PROJECT_ANALYSIS.md` con su corrección.
- **Dos gates.** El gate propio del TP (con los umbrales del enunciado) y el "Sonar way" de SonarCloud son distintos. El plan gratuito no permite configurar el segundo, y eso obligó a escribir el primero. En la comparación de mercado, Codacy ofrece gates configurables gratis para open source: quedó a 8 puntos en la matriz y es una alternativa a probar.

6. Conclusiones

El sistema **se prueba automáticamente en cada despliegue** (smoke y E2E contra Mercado Pago después de cada deploy), **sus integraciones externas están probadas o controladas** (contrato y dobles en cada push, prueba real a mano y E2E automatizado; contingencia documentada para cada dependencia), y **las decisiones de calidad se apoyan en datos** (el gate bloqueó de verdad, la mejora se midió en análisis sucesivos y cada número del informe se puede rastrear hasta su ejecución).

Se cumplen todas las metas del nivel 3. La última fue la **cobertura del código existente del frontend**: estaba parcial (solo líneas sobre el 80 %) y quedó cumplida en las cuatro métricas en el CI #19. El E2E se cumple con un límite explícito: llega hasta crear la preferencia en Mercado Pago, sin el pago.

Trabajo futuro: subir la cobertura de ramas del frontend en los componentes más complejos; una prueba de carga que busque el punto de quiebre; una prueba de concepto de Codacy; conciliar las donaciones pendientes; y que cada ONG cobre en su cuenta de Mercado Pago (Marketplace con OAuth, ya validado hasta la autorización).

7. Anexo

7.1 Entregables y dónde están

Entregable del enunciado	Dónde
Plan de pruebas versionado, con la matriz ISO 25010 y la trazabilidad	`docs/calidad/PLAN_PRUEBAS.md` (v0.9) y `REGISTRO_DEFECTOS.md`
Código de las pruebas, ejecutables con un comando	`backend/test/`, `frontend/src/**/*.test.tsx`, `.github/scripts/`, `load/`; comandos en `COMO_CORRER_PRUEBAS.md`
Pipeline de CI en el repo, con un run en verde y uno fallido real	`.github/workflows/ci.yml`, `deploy.yml`, `load.yml`; verde: CI #15; fallidos reales: CI #4 y CI #7
Reportes automáticos de cobertura, análisis estático y stress	Resumen y artefactos de cada run de CI y de Carga; SonarCloud (sonarcloud.io, proyecto SantiagoPlanasM_voluntariar-PF)
Documentación de la integración externa	`docs/calidad/INTEGRACION_MERCADOPAGO.md`
Registro de defectos completo	GitHub Issues #2–#19 (label `defecto`) y `REGISTRO_DEFECTOS.md`
Documento de la herramienta nueva y matriz de decisión	`docs/calidad/HERRAMIENTA_NUEVA.md`
Métricas interpretadas	`docs/calidad/METRICAS.md`, `TRIAGE_SONAR.md`
Diseño de casos y rojo/verde	`docs/calidad/DISENO_CASOS.md`, `docs/calidad/evidencia/`

7.2 Cómo verificar cada número

- Tests y cobertura: CI #15, `github.com/SantiagoPlanasM/voluntariar-PF/actions/runs/39069697534` (resumen de cada job).
- Stress y esfuerzo: Carga #1, `…/actions/runs/39065266810`.
- Primer deploy automático completo en verde: Deploy #5, `…/actions/runs/39064610099`.
- SonarCloud: historial de análisis y medidas actuales (API pública).
- Defectos: `github.com/SantiagoPlanasM/voluntariar-PF/issues?q=label:defecto`.

7.3 Uso de IA

Se usó un asistente de IA (Claude, en Claude Code) para escribir pruebas, scripts, workflows y documentación, siempre con revisión y con las reglas del proyecto (`CLAUDE.md`, `AI_RULES.md`): tests primero, verificación en SQLite, Postgres y con la zona horaria de Córdoba, mutation testing sobre el código nuevo y una sección de "errores propios" y "no verificado" en cada cambio. Todos los números salen de ejecuciones del proyecto, no de estimaciones del asistente. Las decisiones (nivel, herramientas, pesos de la matriz, qué arreglar y qué aceptar) las tomó el equipo.